
Camera Tutorial

Simple presence-detection recipe

SCMVS – Existence Detection, output and Tool Results

Objective

Create an SCMVS recipe that detects the presence of an object using the **Existence Detection** tool.

Camera IP address: 192.168.100.100

Credentials note: log in to the software with an account that has administrator permissions. Credentials are not included in this tutorial: use those provided by the instructor or laboratory supervisor.

Contents

1	Creating a new recipe	2
2	Initial camera configuration	2
3	Acquiring the Base Image	4
4	Choosing the tool	6
5	Registering the model	7
6	Main Existence Detection parameters	8
7	Testing the model	9
8	Making the inspection more selective	10
9	Configuring the recipe output	12
10	to the runonce command	13
11	Save Image	13
12	Tool Results	14
13	Example of output string composition	16
14	Scheme Management	17

1. Creating a new recipe

Open SCMVS and create a new recipe from scratch by pressing the **Create** button.

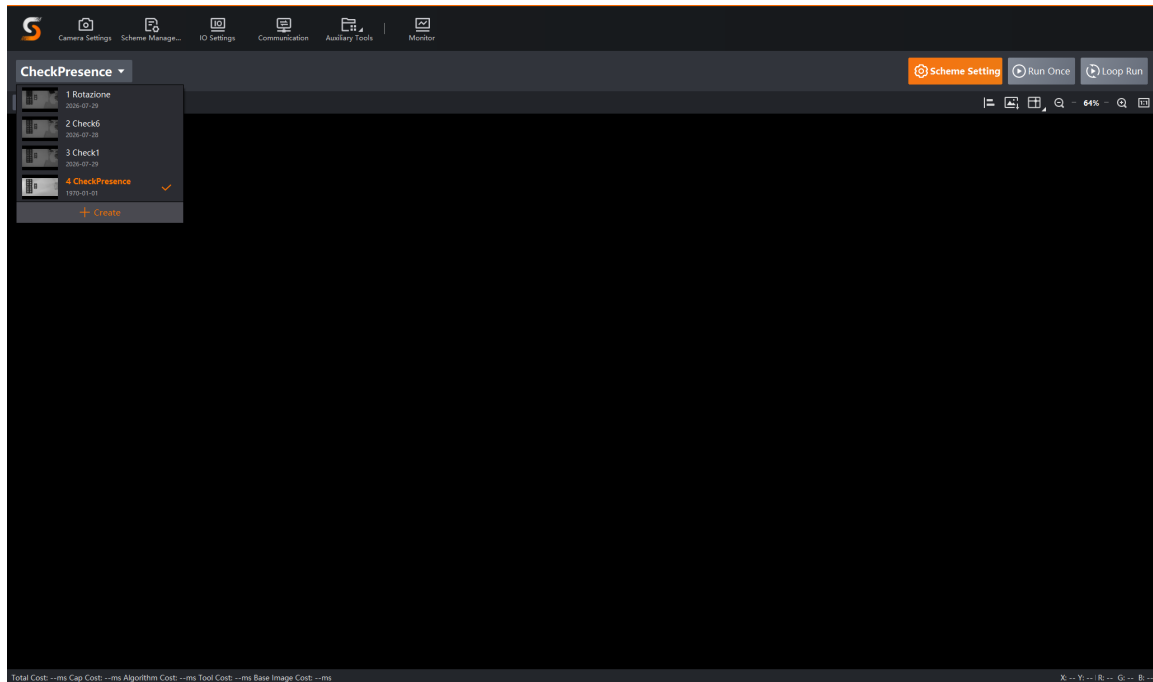


Figure 1: Initial screen for creating a new recipe.

In general, recipe configuration follows this workflow:

Camera → Base Image → Tools → Output

2. Initial camera configuration

Open the recipe and, in the **Camera Param** section, press **One-Key Adjust** to automatically adjust the main acquisition parameters, such as brightness and focus.

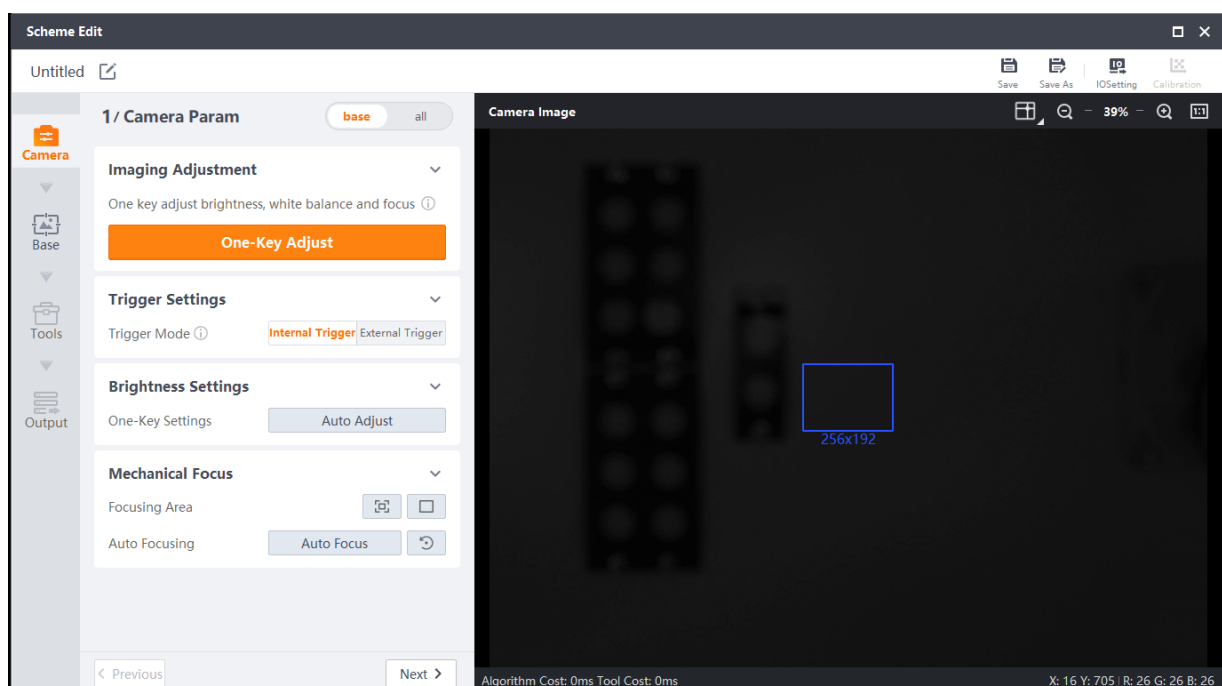


Figure 2: Camera parameter configuration section.

To focus on a specific area of the image, use the blue rectangle in the **Mechanical Focus** section. Position the focus area over the region of interest, as shown in Fig. 3, then repeat the automatic adjustment. The resulting image should be similar to the one shown in Fig. 4.

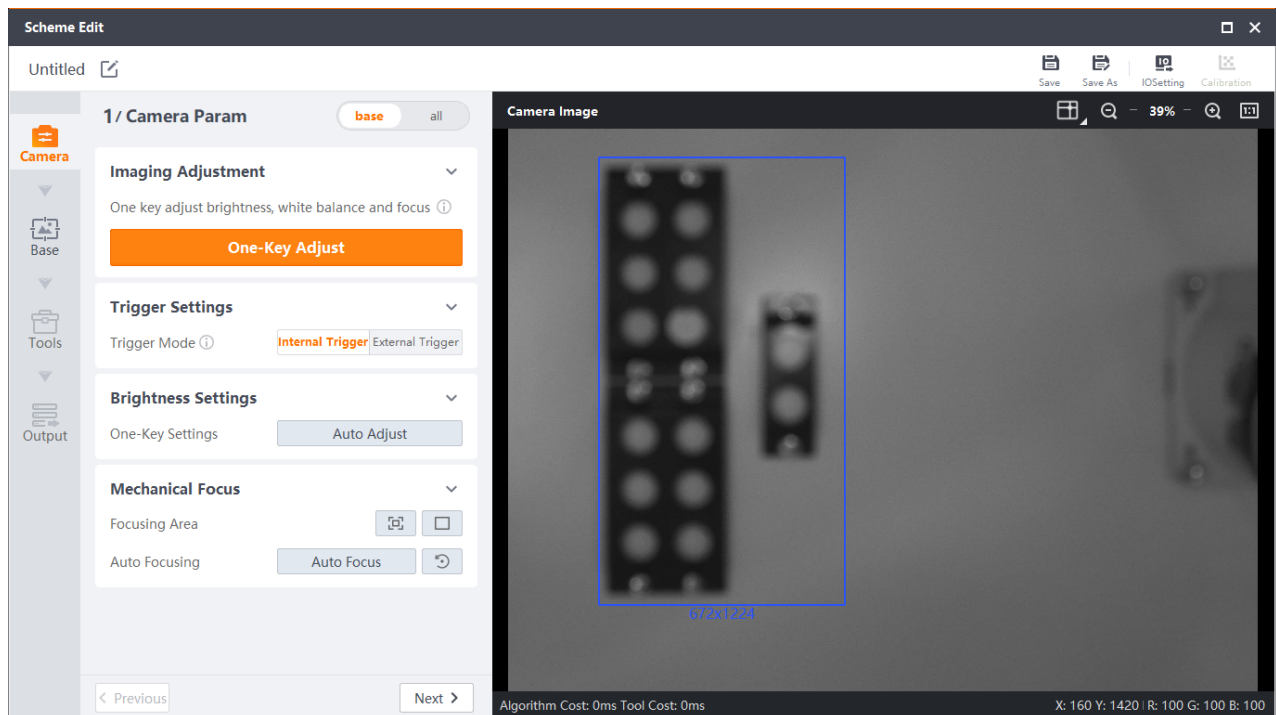


Figure 3: Initial focus area positioned over the region of interest.

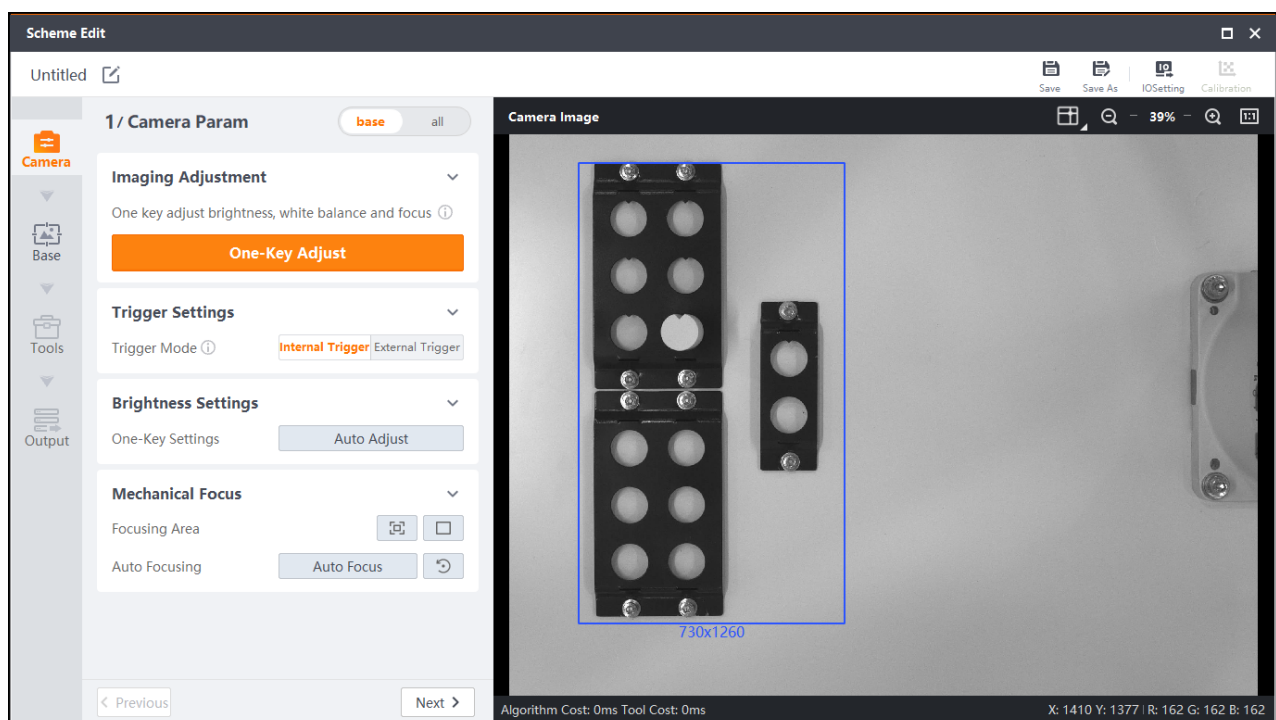


Figure 4: Image obtained after repeating the automatic focus adjustment.

Then set the other options as shown in the example below (Fig. 5). In this case, external **Trigger Mode** with a software trigger is used.

The screenshot displays three sections of a camera configuration interface:

- Trigger Settings:** Includes a 'Trigger Mode' dropdown with 'Internal Trigger' and 'External Trigger' (highlighted in orange), a 'Trigger Source' dropdown set to 'Software Trigger', and an 'Execute' button.
- Brightness Settings:** Features a 'One-Key Settings' button labeled 'Auto Adjust'.
- Mechanical Focus:** Includes a 'Focusing Area' section with two icons (a square with a crosshair and a plain square), and an 'Auto Focusing' section with an 'Auto Focus' button and a circular arrow icon.

Figure 5: Additional camera parameters used in this example.

Note

You can use the **Execute** command to verify that the camera acquires the image correctly. The **all** tab provides access to more advanced parameters. For a first recipe, it is advisable to start with the basic settings and modify advanced parameters only if necessary.

3. Acquiring the Base Image

Press **Next** to move to the **Base Image** section.

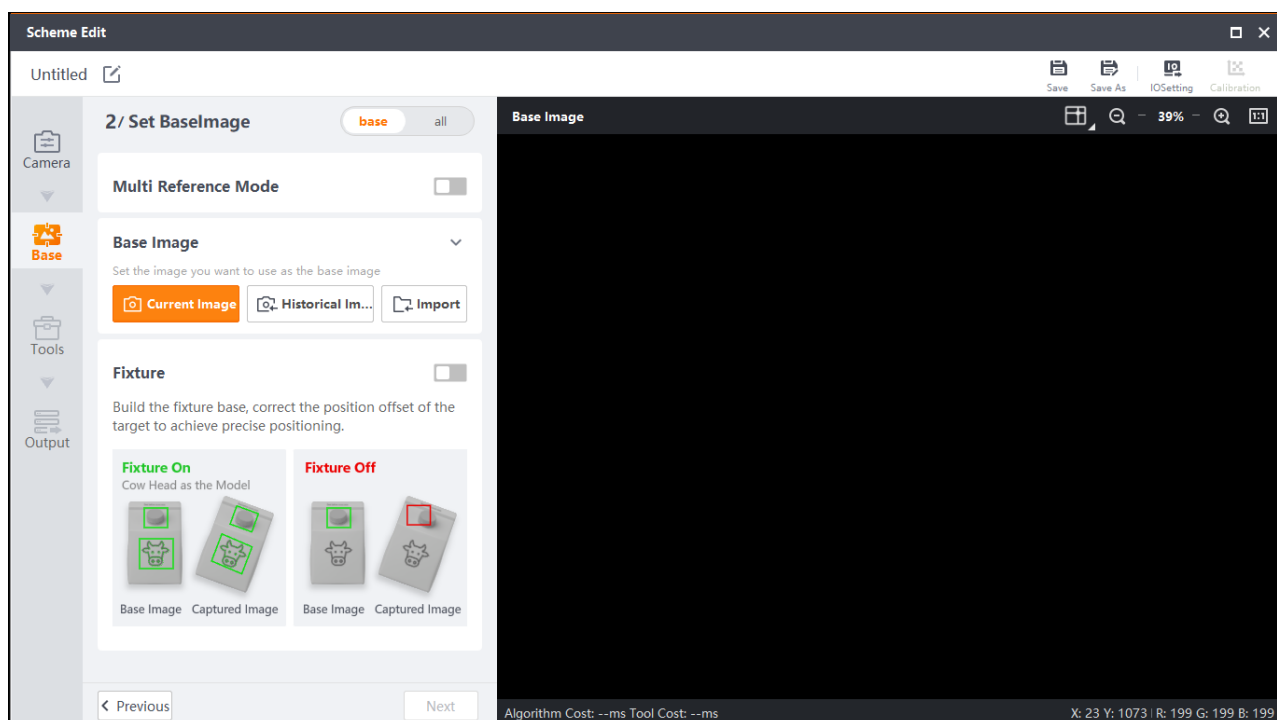


Figure 6: Base Image configuration section.

The **Base Image** is the reference image for the recipe. In this example, we want to detect the presence of an object, so we use as the base image a condition in which the object to be detected is

not present. Select **Current Image**, then acquire the current image using **Capture Image**, then **Next** to proceed to the **Tools** section. Per il momento le sezioni **Multi Reference Mode** e **Fixture** le trascuriamo perché riguardano opzioni aggiuntive non necessarie.

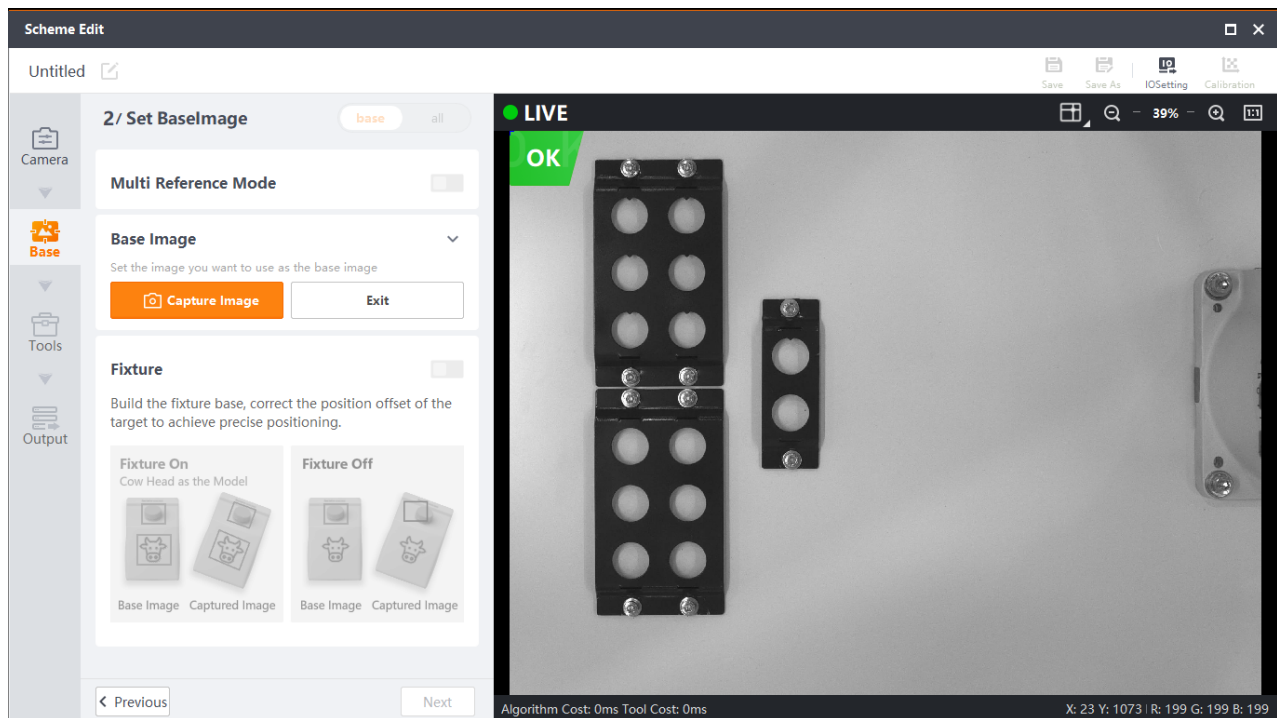


Figure 7: Acquisition of the reference base image.

In the Tools section possiamo scegliere il tool che fa più a caso nostro, ci sono diverse opzioni per svolgere i principali task industriali. Press the + button to display the available tools.

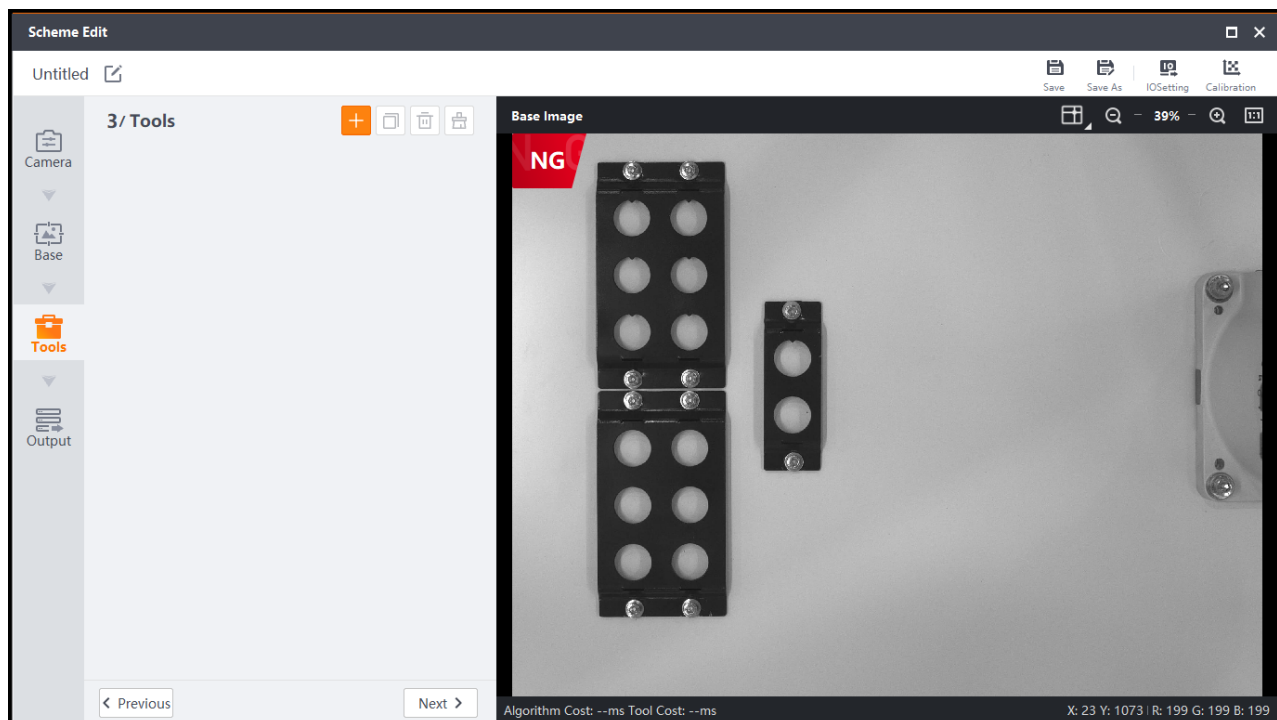


Figure 8: Tools section used to add an inspection tool.

4. Choosing the tool

The software provides many tools that can be used to perform a wide range of tasks.

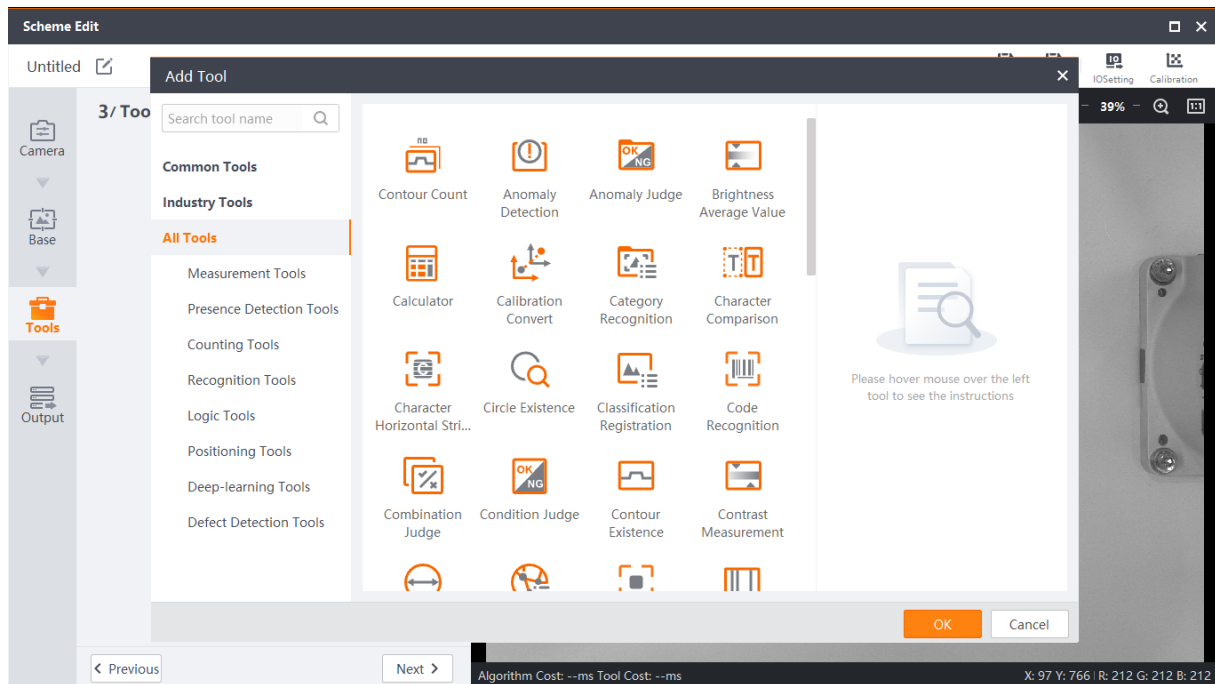


Figure 9: Examples of the inspection tools available in SCMVS.

For this example, we use **Existence Detection**.

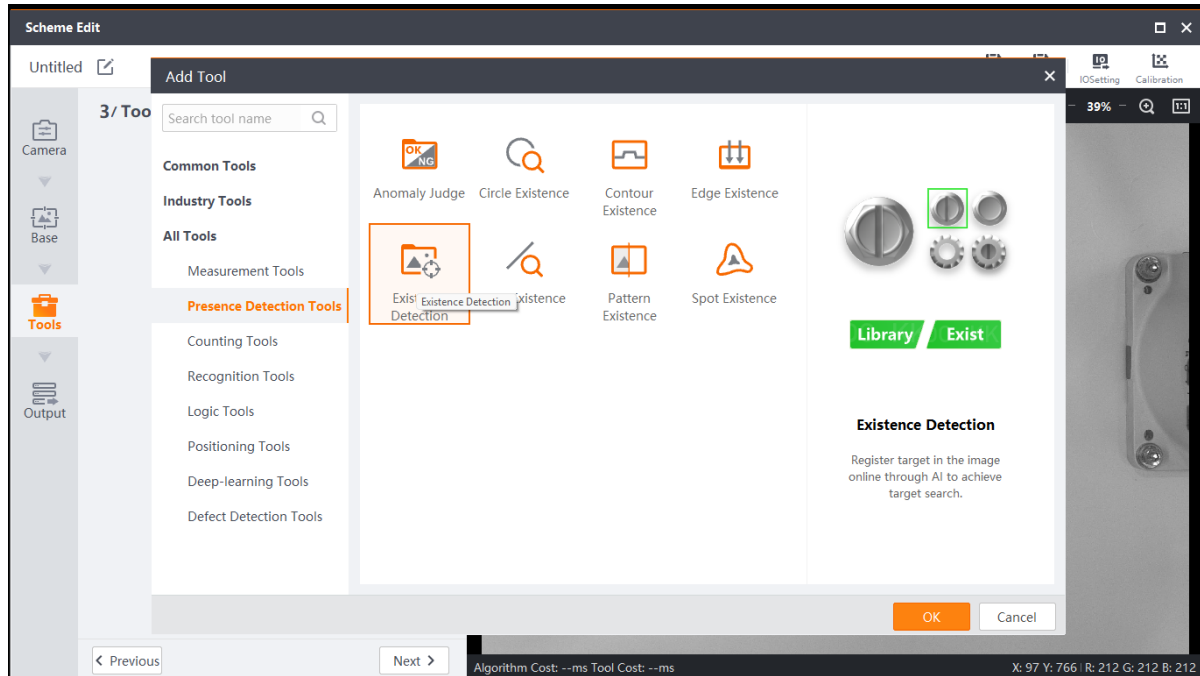


Figure 10: Selection of the Existence Detection tool.

Note

There is not necessarily a single correct tool for every application. The choice depends on the object, lighting, part position, and the type of inspection required.

5. Registering the model

After adding **Existence Detection**, press **Register** to create a new model.

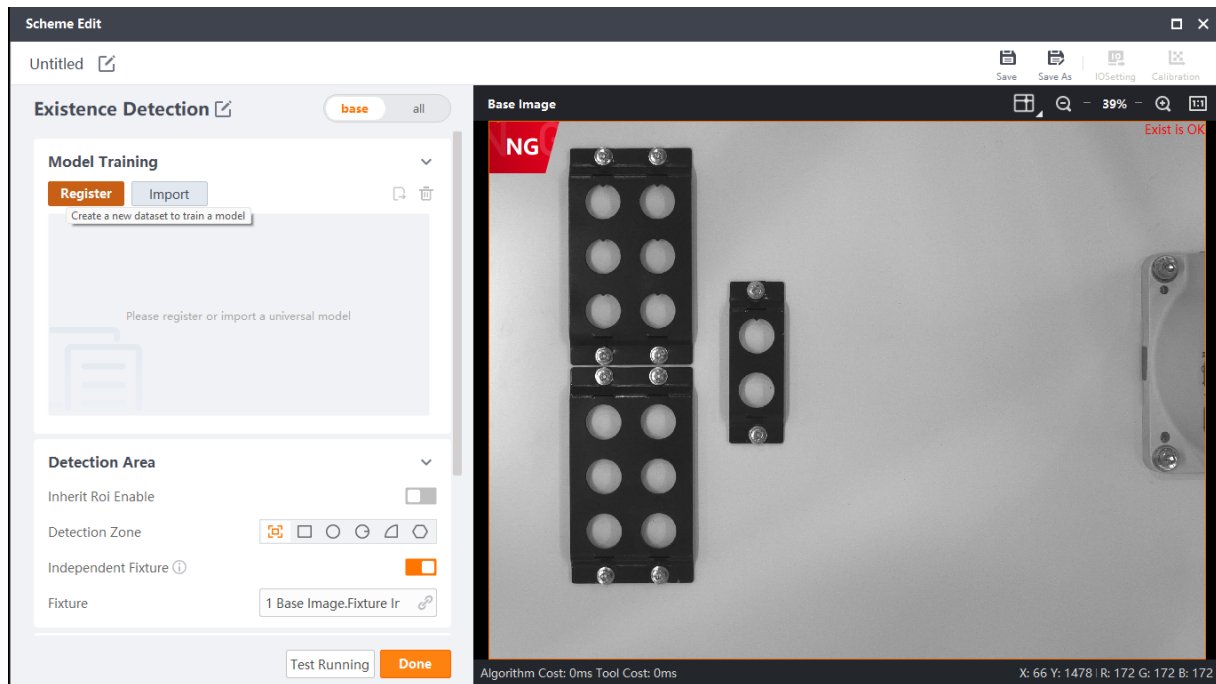


Figure 11: Initial screen of the selected Existence Detection tool.

For a simple example, prepare at least two images:

- one without the object;
- one with the object to be detected.

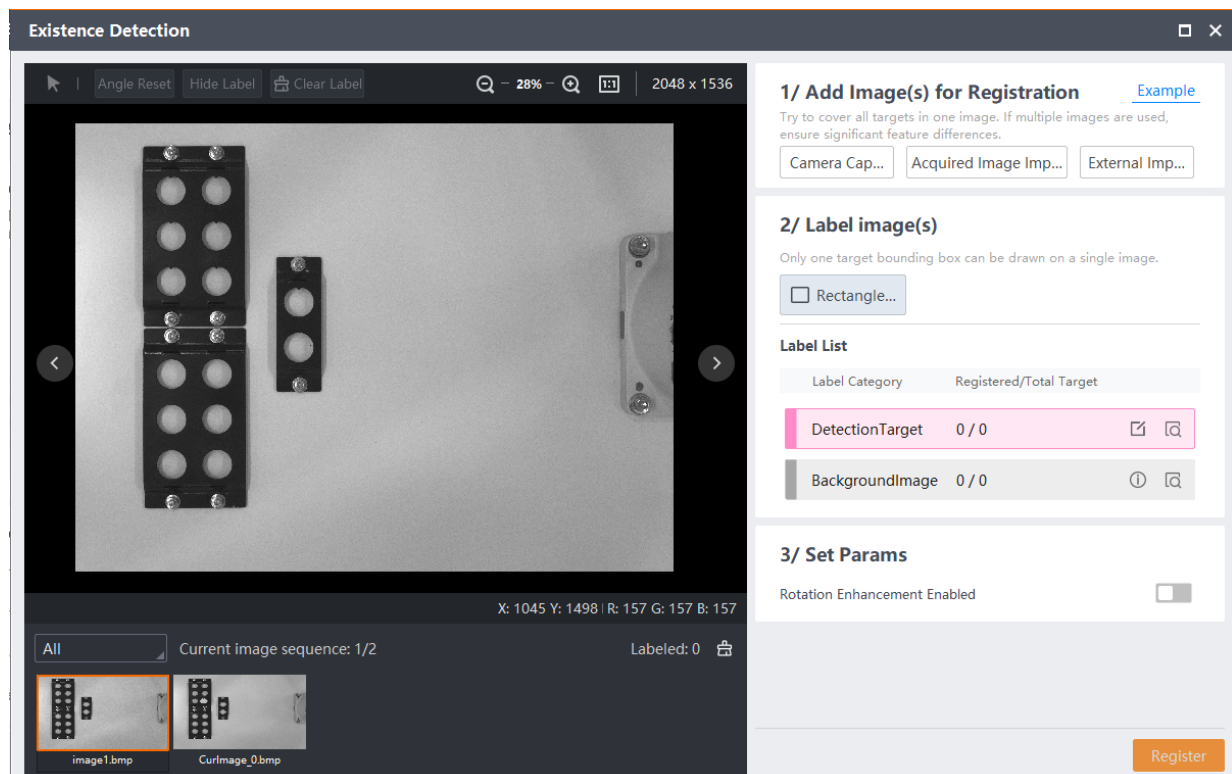


Figure 12: Image acquisition screen used to register a new model.

These images allow the system to learn both the **background** and the **target**. Use the image without the object as the **Background Image** and the image with the object as the labeled image. To label it, draw a rectangle around the object using the selection tool, as shown in Fig.13 . Additional images can be added to improve detection robustness.

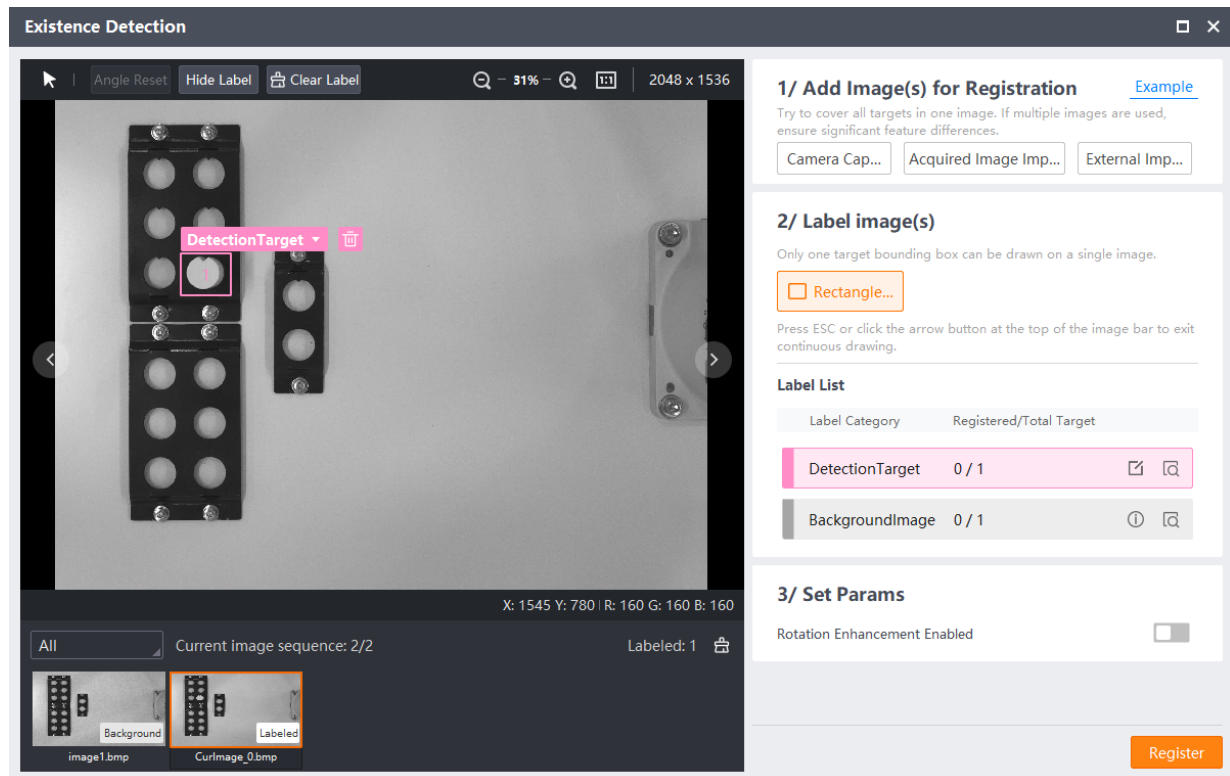


Figure 13: Labeling of the acquired images used to train the model.

6. Main Existence Detection parameters

Once registration is complete, the tool provides several important settings.

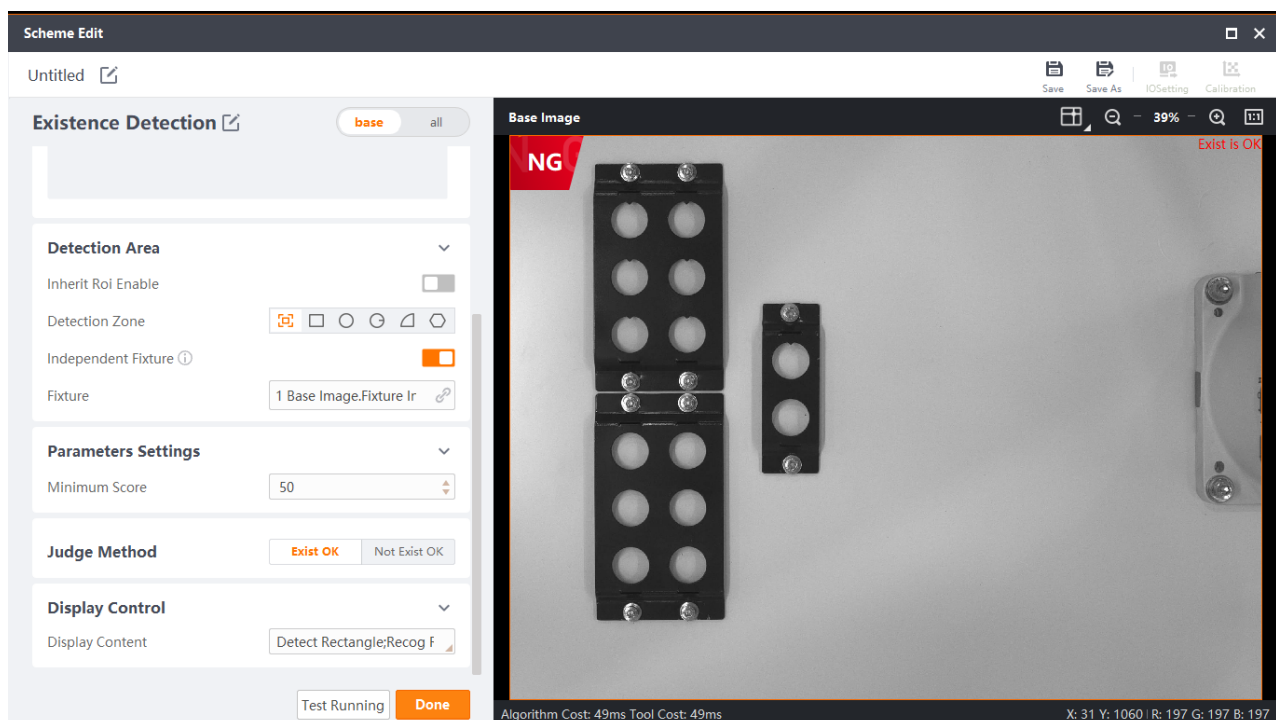


Figure 14: Additional settings available for the Existence Detection tool.

Detection Area

The **Detection Area** defines the region of the image in which the system should search for the target. If no specific area is set, the model may search for the object over a very large portion of the image.

Minimum Score

The **Minimum Score** parameter sets the minimum similarity threshold required for a detection to be considered valid.

Judge Method

In this example, select:

Exist OK

The logic is therefore:

```
Object detected -> OK  
Object not detected -> NG
```

The **Not Exist OK** option reverses the logic.

7. Testing the model

To verify the behavior of the tool, open **Test Running** mode. Run several tests by moving or removing the object. Figure 15 compares the tool behavior when the target object is present and when it is absent.

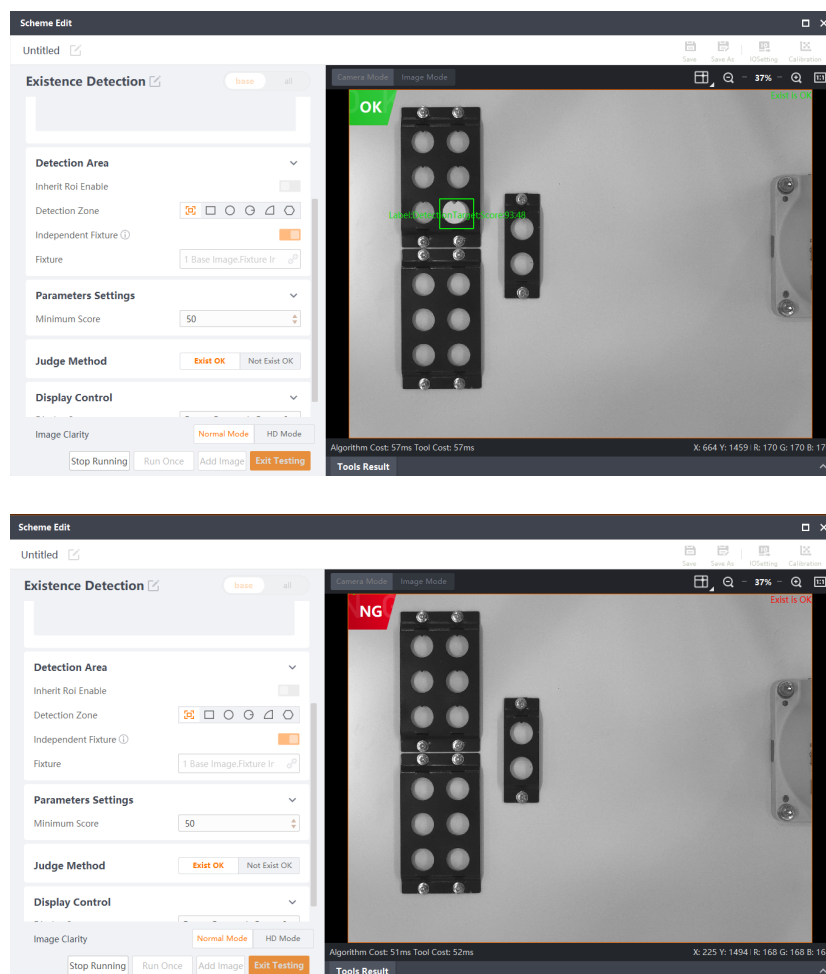


Figure 15: Comparison of the tool behavior when the target object is present and when it is absent.

If no **Detection Area** has been set, the model may detect the target even in a position different from the desired one, as shown in the following images.

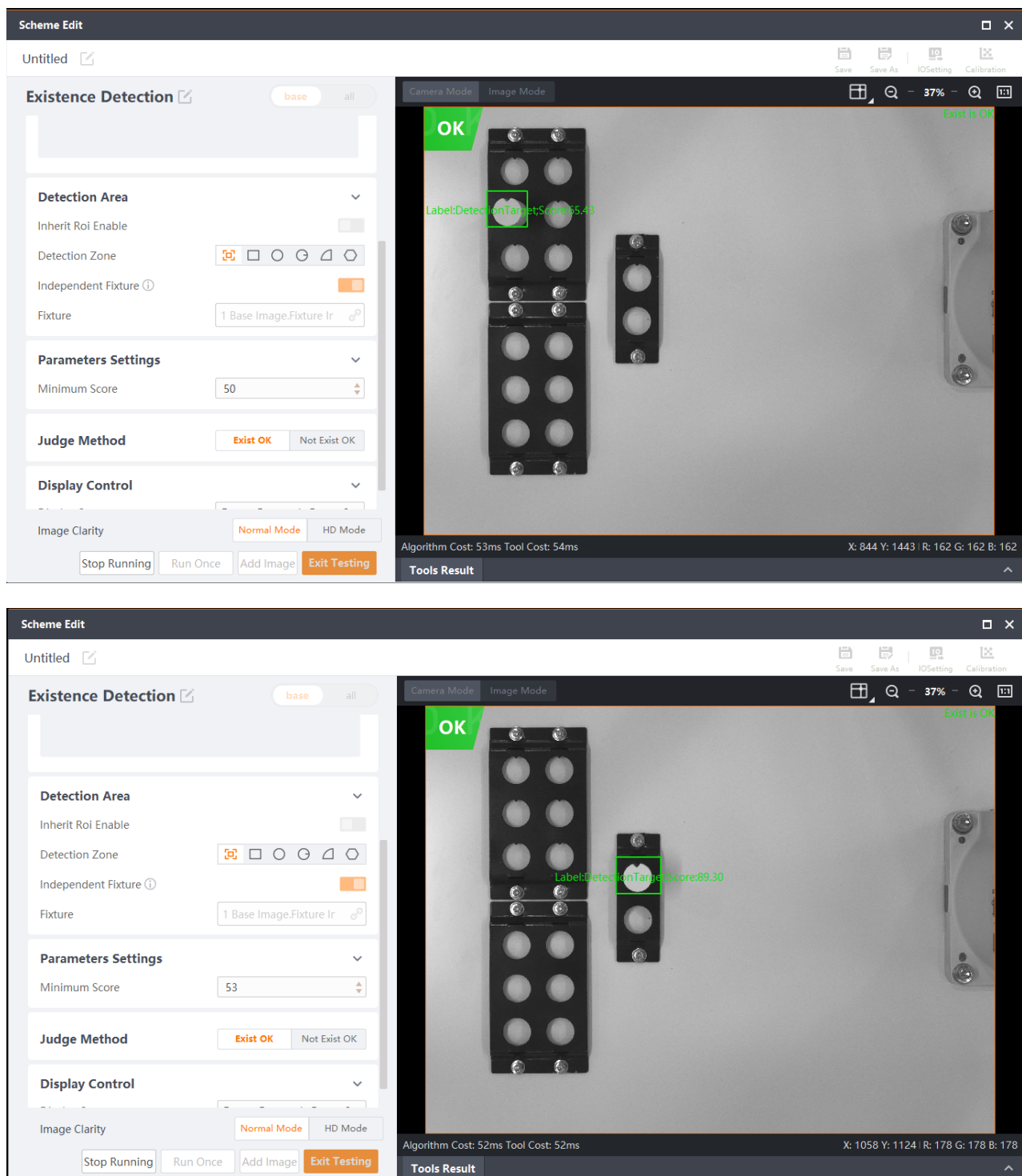


Figure 16: Examples of the target being detected even when it is outside the desired position.

This can happen because the model searches for features similar to the object throughout the available area.

8. Making the inspection more selective

If the system detects the object in undesired positions, several strategies are possible:

- increase the **Minimum Score**;

- add more positive and negative images to the model;
- restrict the search region using the **Detection Area**.

For this example, we therefore use a rectangular **Detection Area**, as shown in the following example (Fig. 17). This tells the vision system to search for objects only in the region of interest and resolves the problems encountered previously.

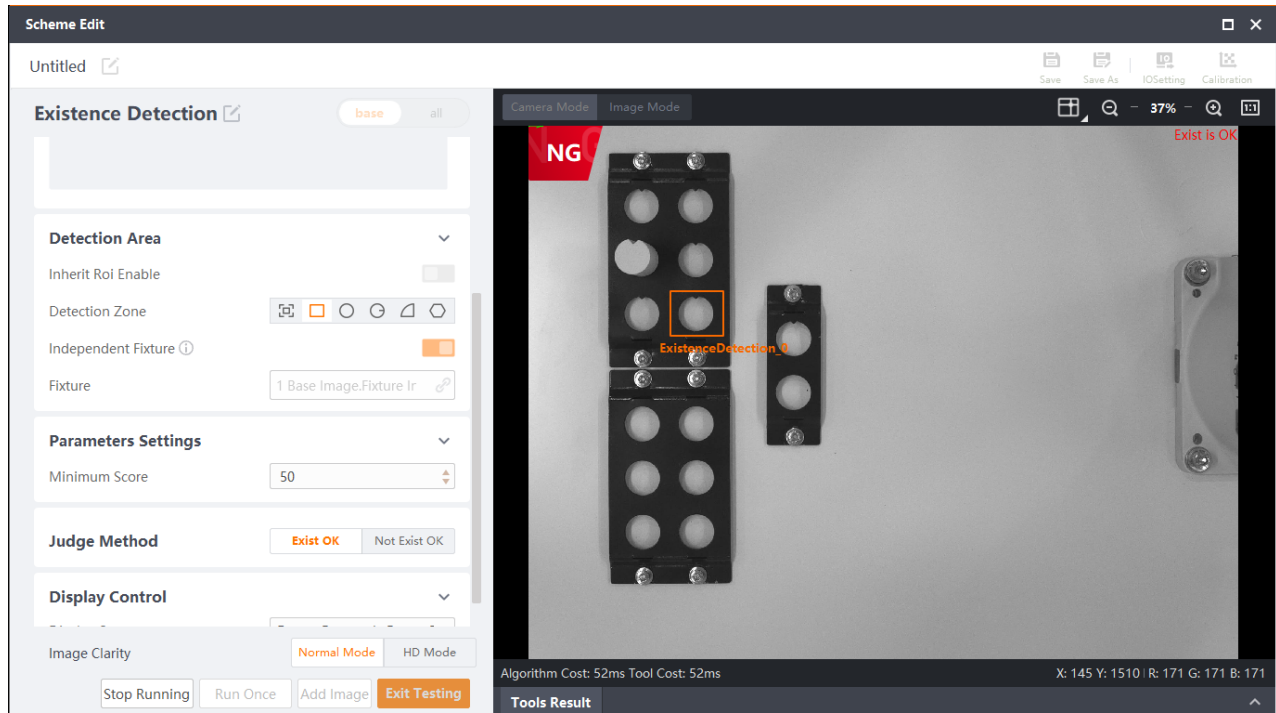


Figure 17: Definition of a rectangular Detection Area around the region of interest.

After restricting the detection area, the previous false-positive case is now correctly classified as NG (Fig. 18). At the same time, the valid detection case remains unaffected (Fig. 19).

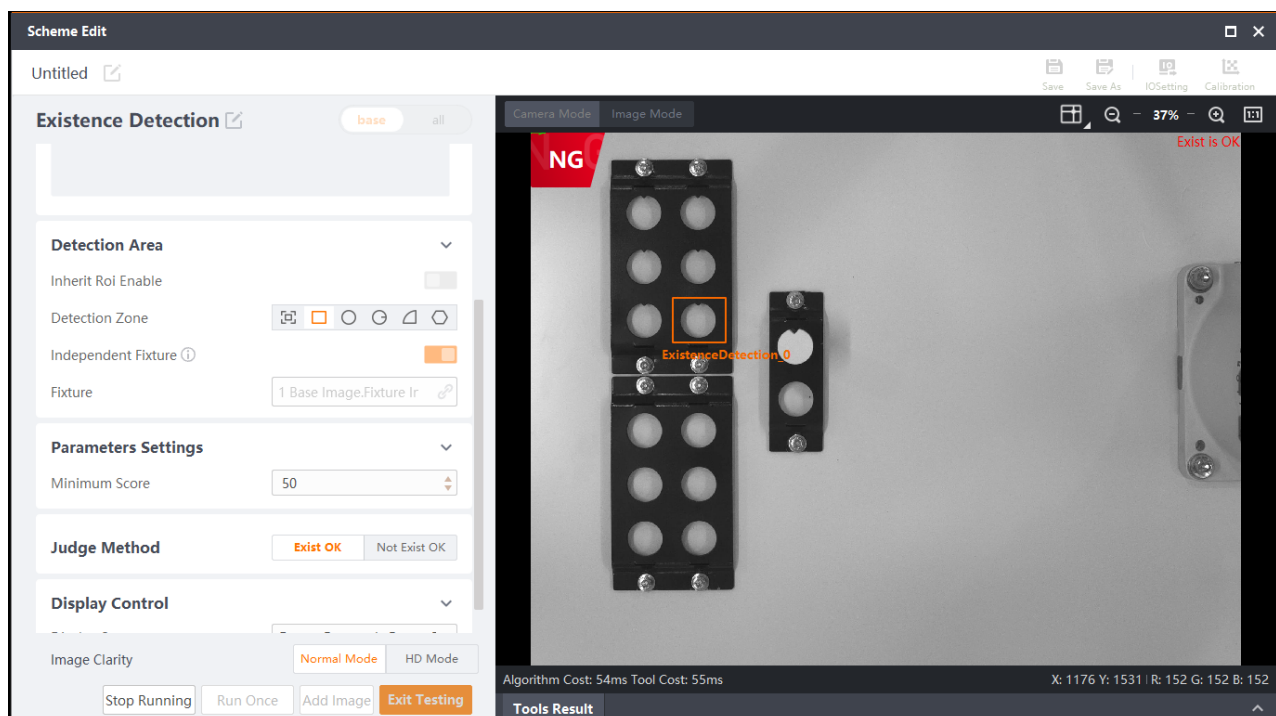


Figure 18: The previously observed false positive is correctly rejected after restricting the Detection Area.

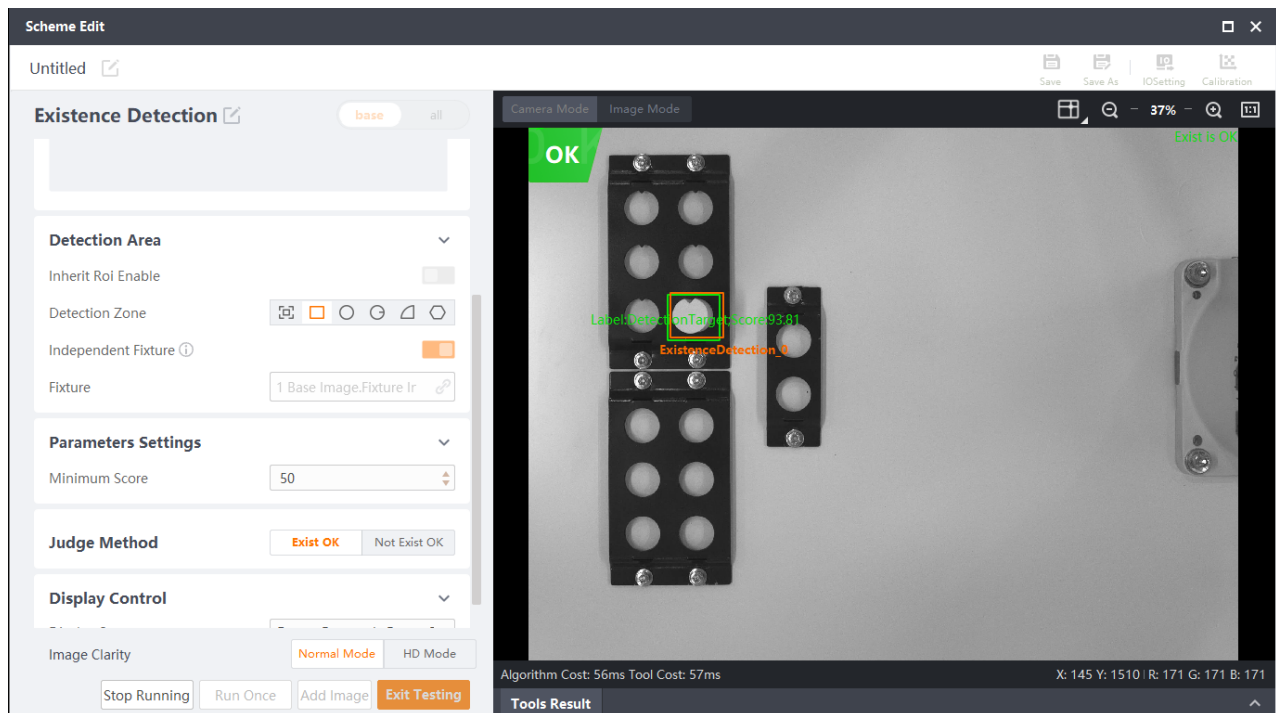


Figure 19: The valid detection case remains correctly classified after restricting the Detection Area.

9. Configuring the recipe output

Once testing is complete, exit testing mode and move to the **Output** section.

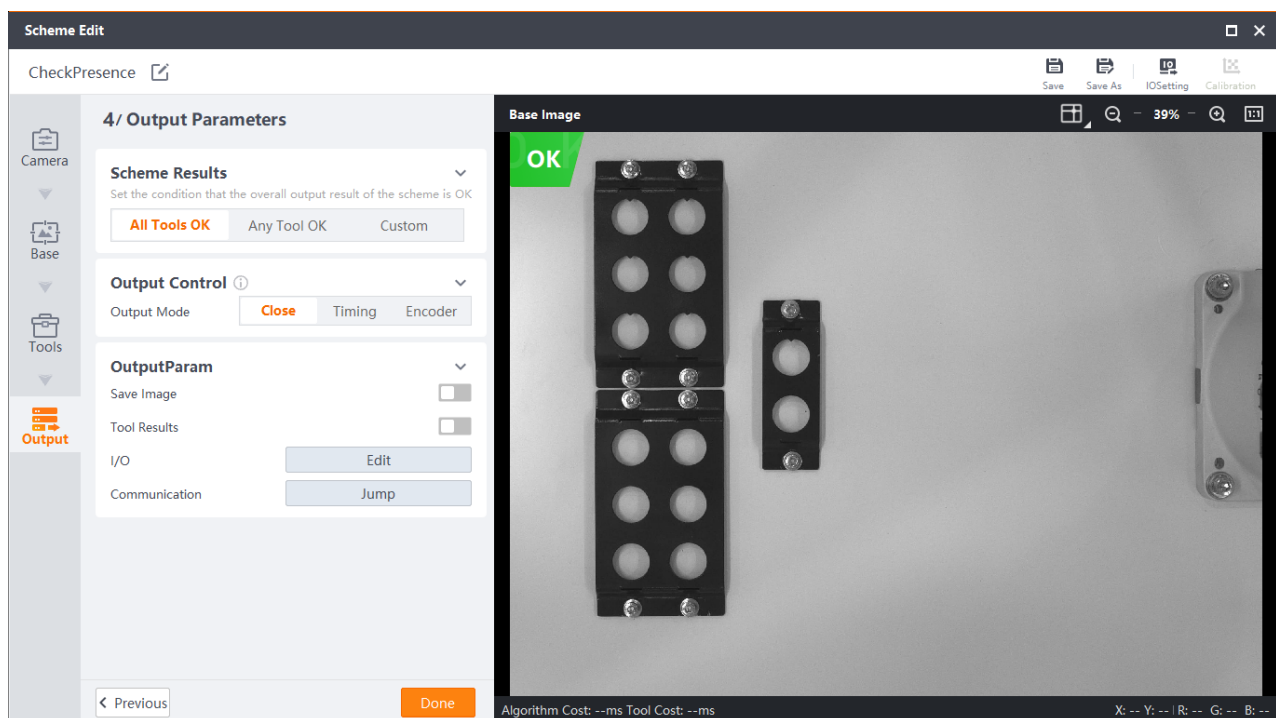


Figure 20: Output configuration section of the recipe.

For this simple recipe, the following setting can be used:

```
Scheme Results = All Tools OK
```

This means that the overall recipe result is OK only when all tools in the recipe return OK.

With **Output Control = Close**, no timed or encoder-related output modes are being used.

10. to the runonce command

The recipe can be executed via communication using the **runonce** command:

```
SocketSendStr(P:Com_Visione,"runonce",P:I_StatoMess)
```

In **Communication Control**, the command is associated with:

```
CameraExecute = RunOnce
InputCommand = runonce
ReturnExecuteOK = OK
ReturnExecuteNG = NG
```

Communication Control

Add Action

Index	CameraExecute	EncodeFormat	InputCommand	InputParam1	InputParam2	InputParam3	ReturnExecuteOK	ReturnExecuteNG	OutputValueEna	OutputValue	Oper
1	RunOnce	ASCII	runonce	Nothing	Nothing	Nothing	OK	NG	False	[	
2	ContinuousRun	ASCII	startloop	Nothing	Nothing	Nothing	OK	NG	False		
3	StopRun	ASCII	stoploop	Nothing	Nothing	Nothing	OK	NG	False		

Sample

CameraExecute	EncodeFormat	InputCommand	InputParam1	InputParam2	InputParam3	ReturnExecute...	ReturnExecute...	OutputValue...	OutputValue...	OutputValue...	OutputValue...
RunOnce	HEX	72 75 6E 6F ...	Exposure Ti...	Gain (dB)	Nothing	4F 4B	4E 47	Yes	5B	26	5D

1. Peer request (send command in ASCII code): runonce 500,6
2. Camera execution OK reply: OK[500&6]
3. Camera execution NG reply: NG[500&6]

4. Note: If you need to use the SCMVS client to view the camera operation results, the communication control function needs to be used in camera mode. Before use, manually enable loop operation in the client, otherwise you may not be able to receive the camera operation results. After use, manually turn off loop operation to avoid affecting normal function use. The cyclic command is limited to use under external triggering, and the triggering source is communication triggering or button triggering!

Save

Figure 21: Communication Control configuration screen.

If **Tool Results** is disabled, the OK/NG response configured in the Communication Control refers only to the execution status of the **runonce** command, not to the result of the visual inspection. Therefore, if the image acquisition and the **runonce** command are executed successfully, the camera may return:

OK

even when the object is not present and the vision recipe itself evaluates the inspection as **NG**.

Warning

Therefore, **OK** must not automatically be interpreted as “part present”, nor **NG** as “part absent”. To determine the actual result of the visual inspection, the tool result must be explicitly transmitted through **Tool Results**.

11. Save Image

The **Save Image** function allows images acquired by the camera to be saved. In this case, it can be left disabled to avoid unnecessarily using the camera’s internal memory.

12. Tool Results

The **Tool Results** function allows a more detailed output to be built. Enable Tool Results (Fig. 22) and press **Edit** next to “Unfinished” to open the configuration screen shown in Fig. 23.

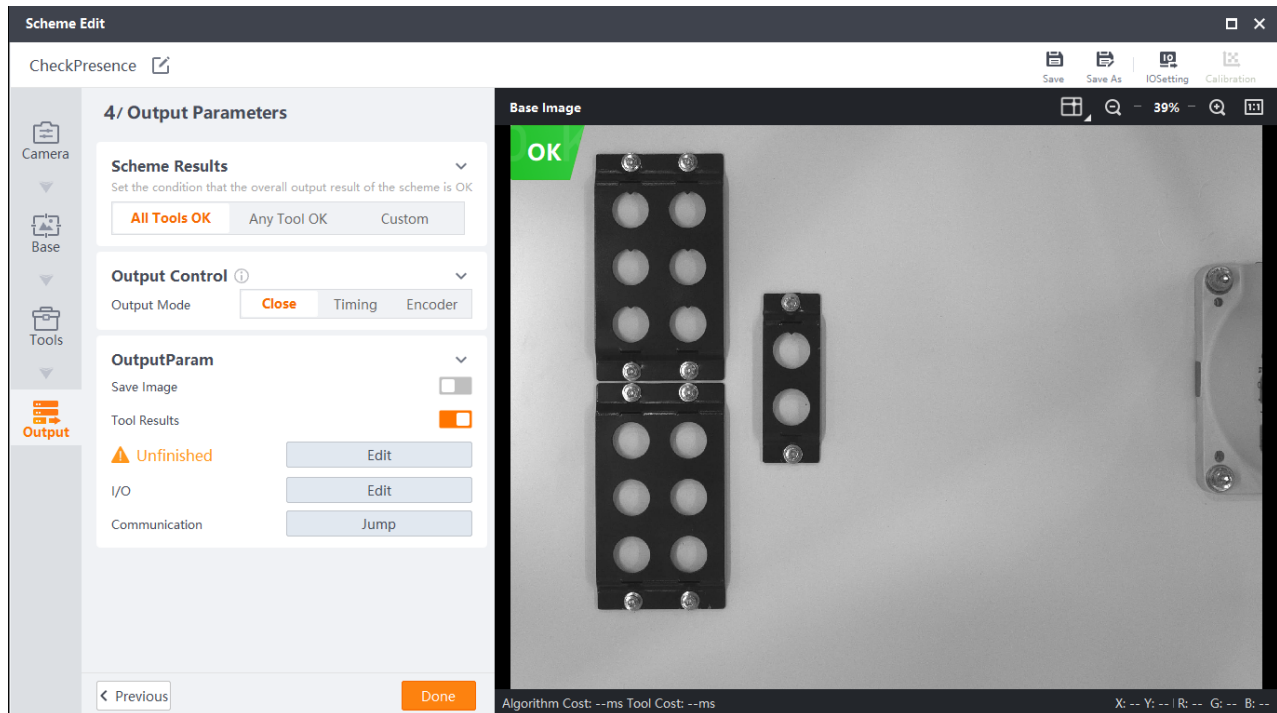


Figure 22: Tool Results activation screen.

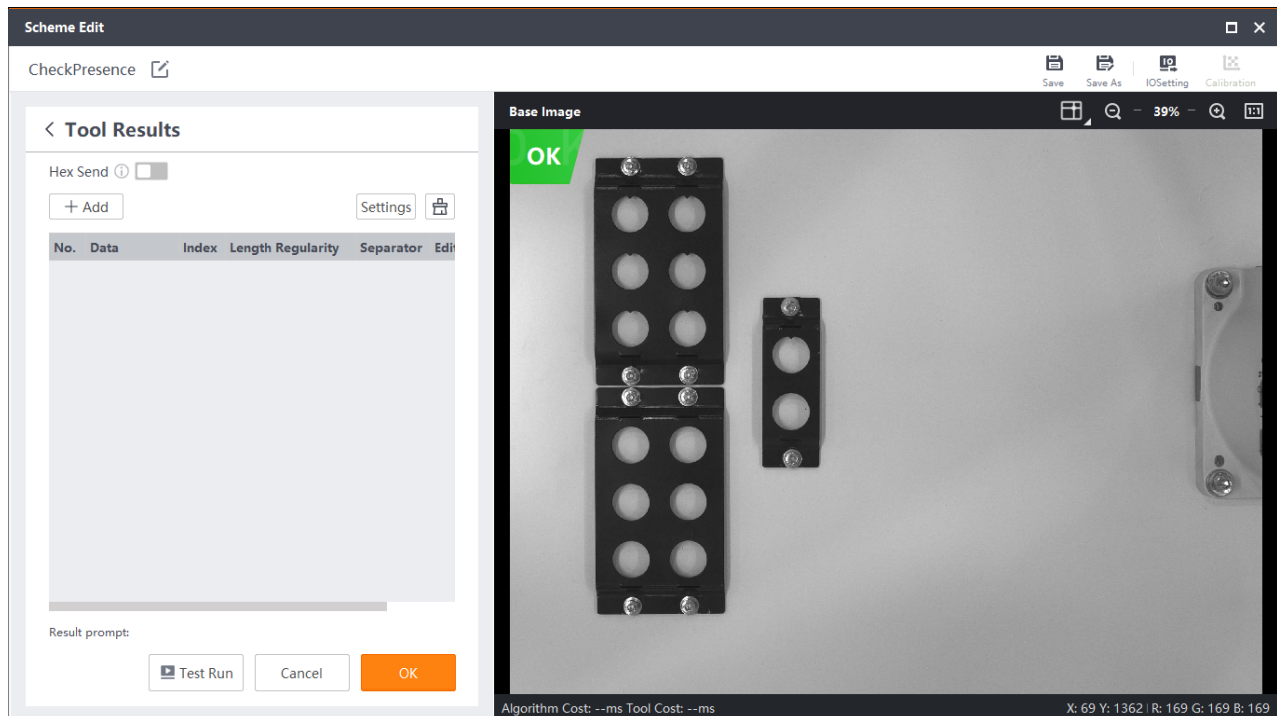


Figure 23: Tool Results configuration screen.

The software allows you to add:

- **Data:** a value produced by one of the recipe modules;
- **Symbol:** a symbol or separator;

- **Array:** a set of data;
- **Custom:** manually defined text.

By pressing:

+ Add -> Data

the different recipe modules are displayed.

In this example, entries such as the following may appear:

```
0 Camera Image
1 Base Image
3 Project Results
6 Existence Detection
```

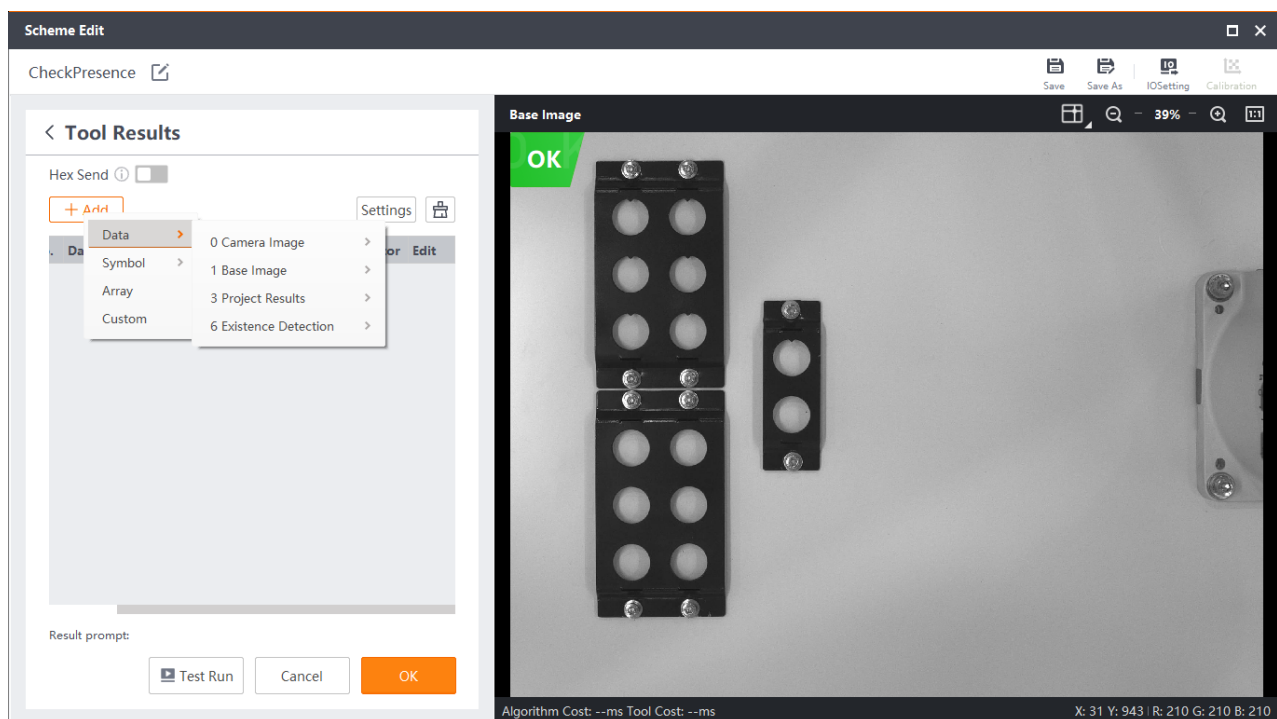


Figure 24: Recipe modules available when adding a Data field.

The numbers identify the modules within the recipe. In particular:

- **Camera Image** contains data related to image acquisition;
- **Base Image** contains data related to the reference image;
- **Project Results** contains the overall recipe result;
- **Existence Detection** contains the data produced by the presence-detection tool.

By selecting **Existence Detection** (Fig. 25), you can choose data such as:

```
Module Status
Module Status String
Confidence
...
```

For a simple presence check, the two most immediate values are **Module Status** and **Module Status String**.

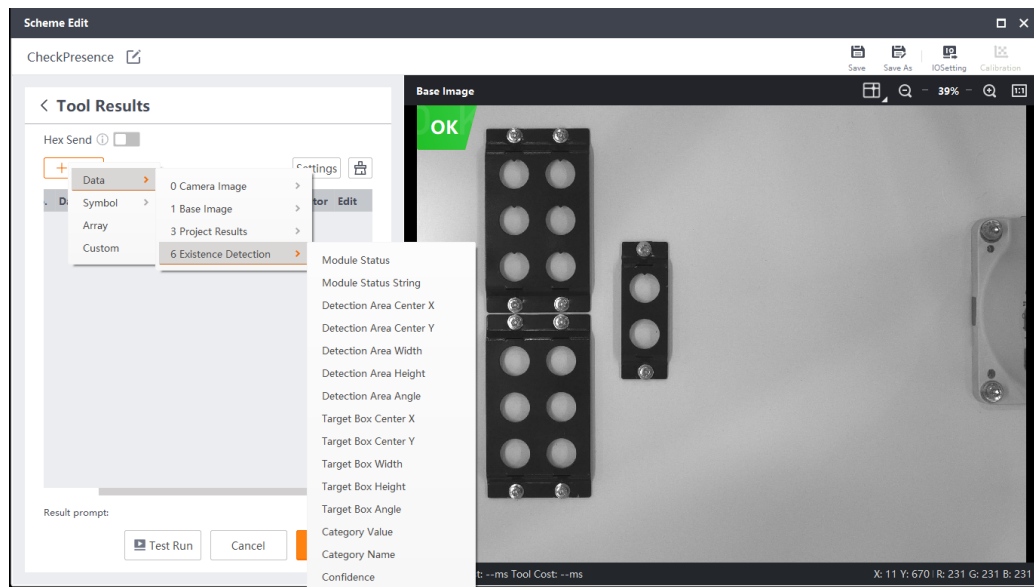


Figure 25: Data fields available for the Existence Detection tool.

The logic is:

```
Object detected:
Module Status = 1
Module Status String = OK
```

```
Object not detected:
Module Status = 0
Module Status String = NG
```

13. Example of output string composition

We can create an output string composed of the following fields, using ; as the separator:

```
Custom -> PRESENCE
Module Status -> 1 or 0
Module Status String -> OK or NG
```

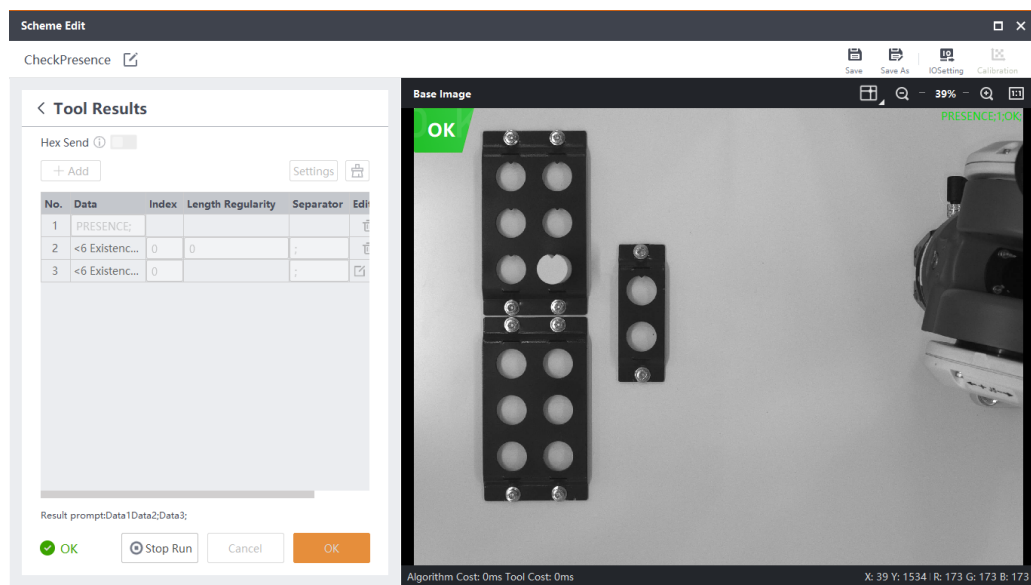


Figure 26: Example of a custom output string built with Tool Results.

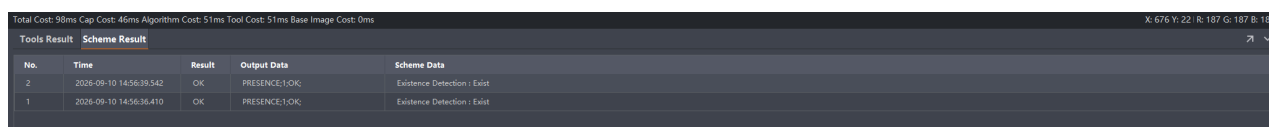
The result will be, for example:

```
PRESENCE;1;OK;
```

or:

```
PRESENCE;0;NG;
```

During testing, the composed string can be checked in the **Result prompt** section, at the top right of the image (for example, shown in green in Fig. 26), and, during recipe execution, in the **Output Data** column of the **Scheme Result** section, situated in basso al software.



No.	Time	Result	Output Data	Scheme Data
2	2026-09-10 14:56:39.342	OK	PRESENCE;1;OK;	Existence Detection : Exist
1	2026-09-10 14:56:36.410	OK	PRESENCE;1;OK;	Existence Detection : Exist

Figure 27: Scheme Result section showing the generated output data.

When the recipe is started from code using `runonce`, the response configured in **Communication Control** is sent together with the data produced by **Tool Results**. The execution-status response and the inspection data are therefore concatenated into a single string returned to the robot.

With the configuration shown in the example:

```
ReturnExecuteOK = OK
Tool Results = PRESENCE;1;OK;
```

the robot can therefore receive:

```
OKPRESENCE;1;OK;
```

Here, the initial OK confirms that `runonce` was executed successfully, while `PRESENCE;1;OK;` contains the actual result generated by the vision tool. This makes it possible to distinguish the execution status from the result of the visual inspection.

This functionality is especially useful when the robot needs more information than a simple OK/NG response, since additional values such as module status, confidence, or coordinates can also be included.

Note

To determine exactly which string is returned by the vision system, the following lines of code can be used:

```
// Read the response
SocketReadStr(P:Com_Visione,P:S_RispostaVisione,10000,P:I_StatoRisposta)

// Print the vision-system response on the TP.
SetRtInfo(P:S_RispostaVisione.value,70003)
```

This makes it possible to display the complete output string directly on the TP, which is useful during testing and debugging.

14. Scheme Management

Once the recipe configuration is complete, it is important to configure **Scheme Management** correctly so that the robot can change recipes through TCP communication.

To access recipe management, open the **Scheme Management** section from the top menu. This screen displays all recipes available in the system and allows the settings for remote recipe selection to be configured.

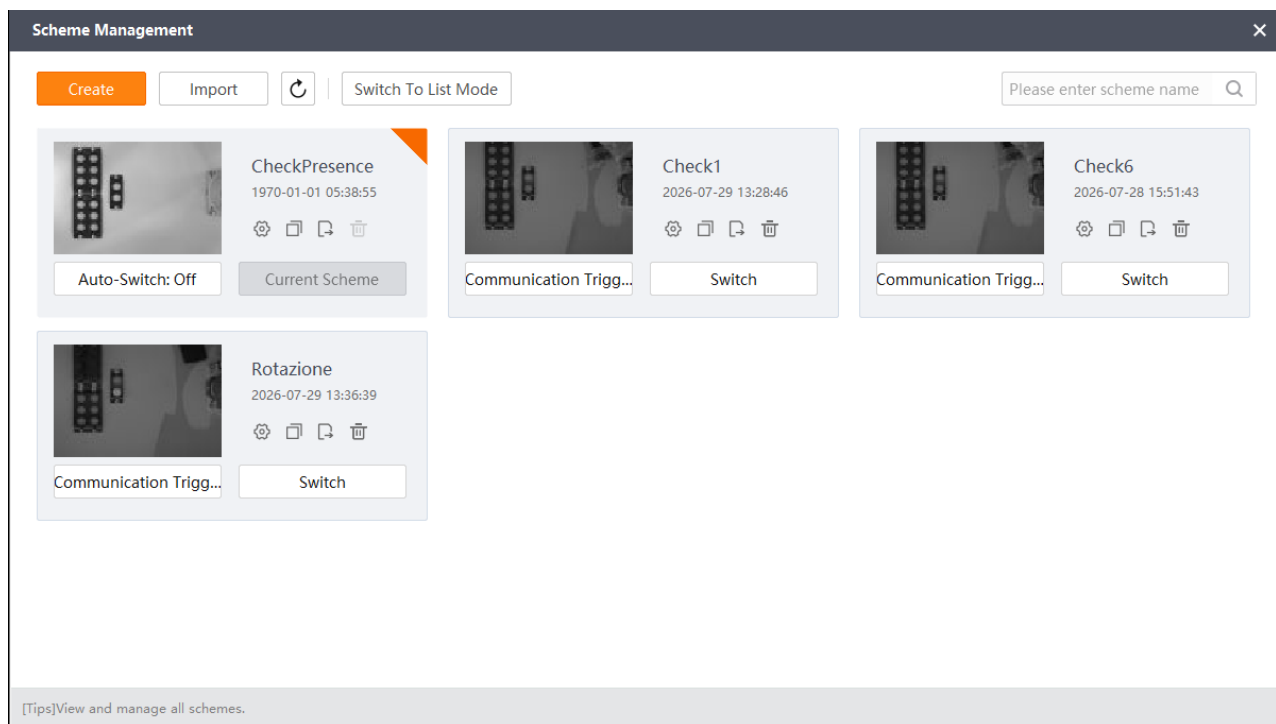


Figure 28: Scheme Management section for recipe selection and switching.

As shown in the image, the newly created recipe initially displays:

Auto-Switch: Off

In this condition, the recipe can be selected and executed manually from SCMVS, but it cannot be called automatically by the robot via TCP communication.

To enable remote recipe switching, press **Auto-Switch** and select:

TriggerCommunication

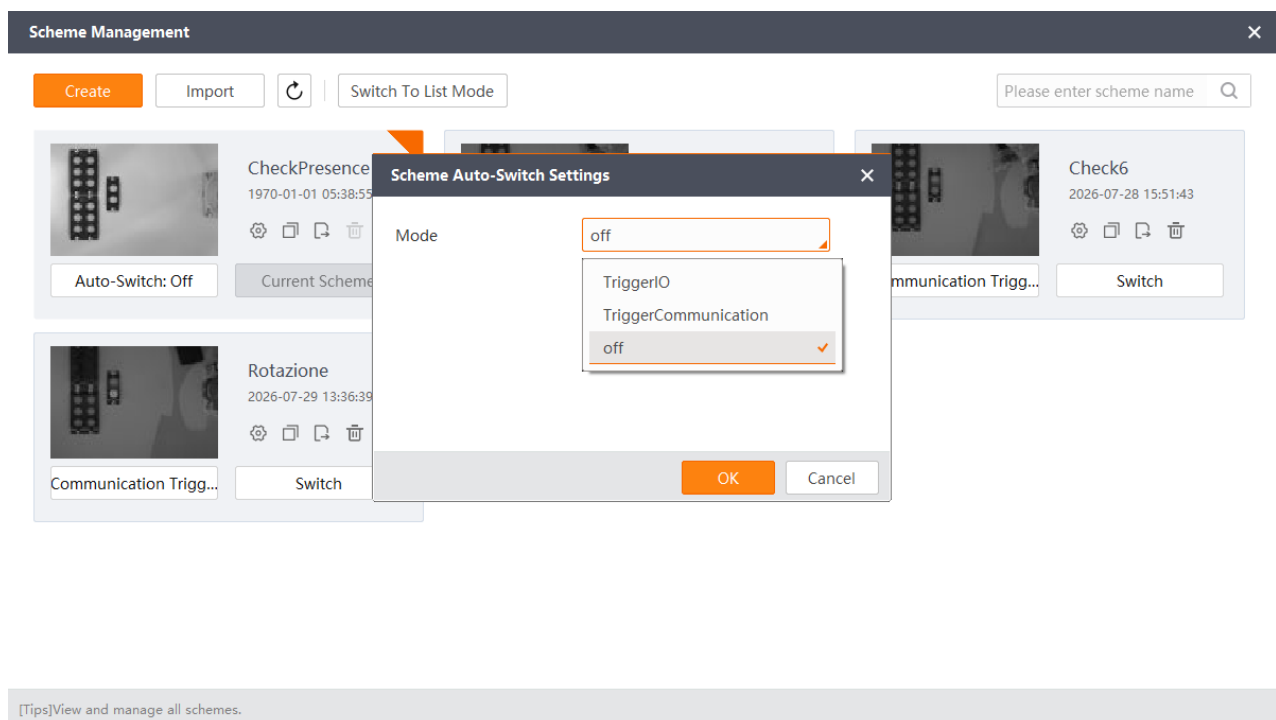


Figure 29: Auto-Switch configuration options.

At this point, SCMVS displays the parameters related to recipe switching via communication.

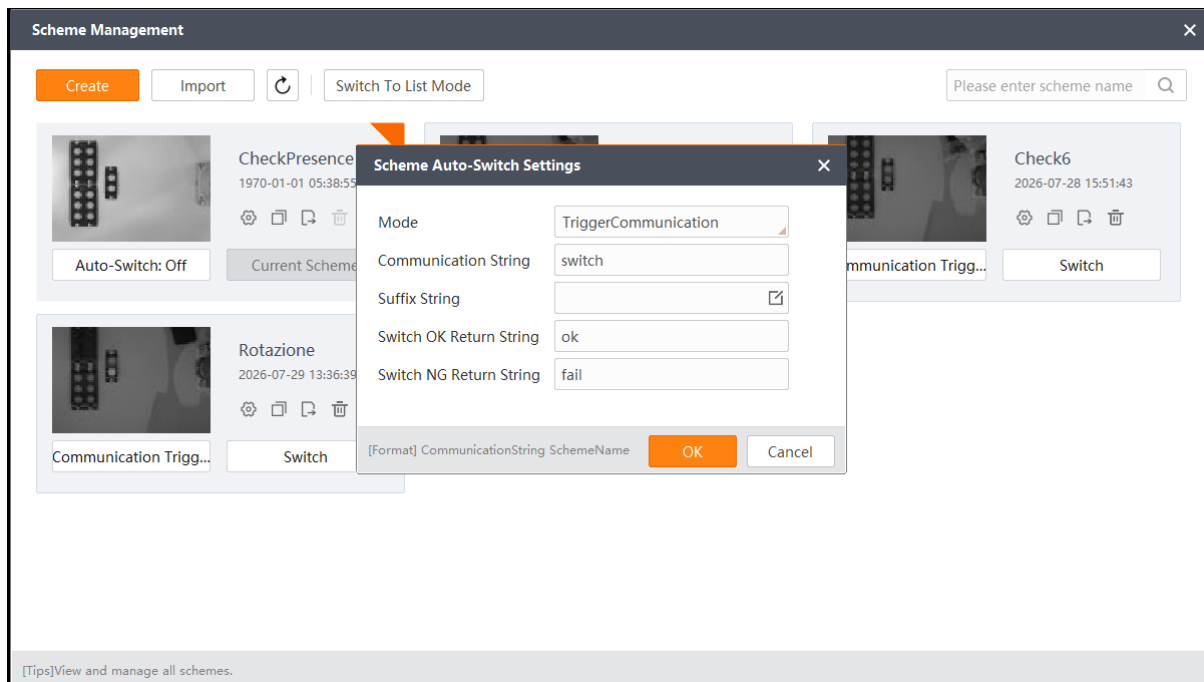


Figure 30: TriggerCommunication settings for automatic recipe switching.

The format required by SCMVS is:

```
CommunicationString SchemeName
```

Therefore, if the recipe is called **CheckPresence**, to correctly send the signal to the robot using the following instruction:

```
SocketSendStr(P:Com_Visione,"scheme CheckPresence",P:I_StatoMess)
```

the parameters must be modified as follows:

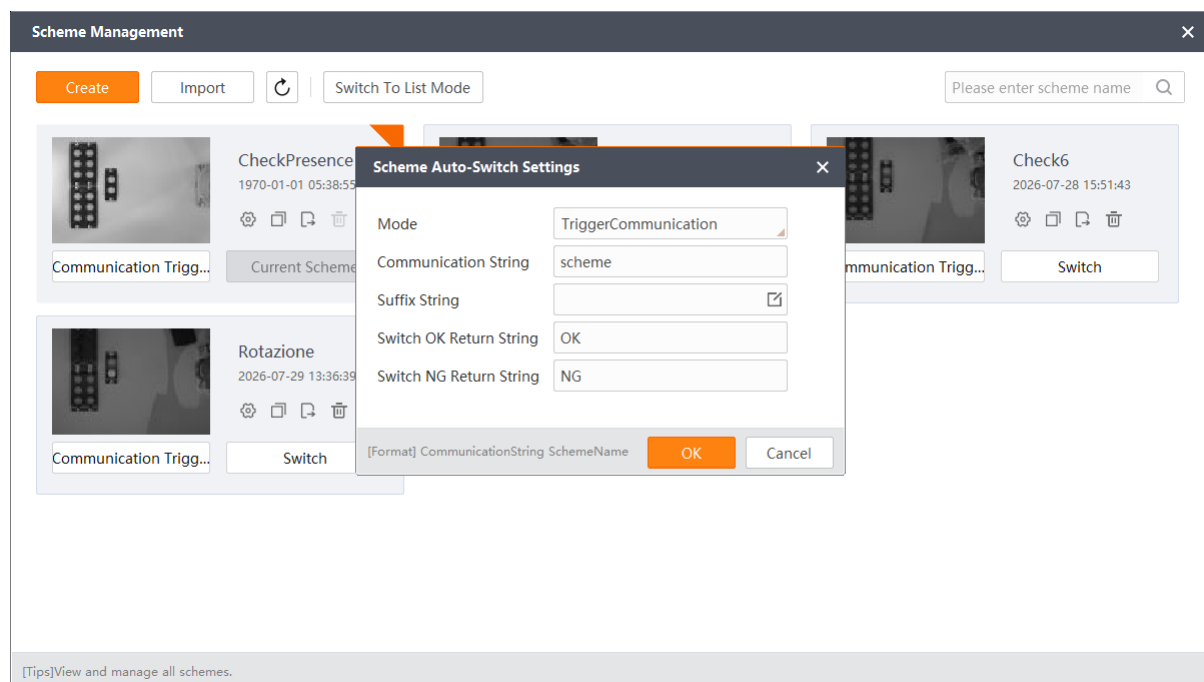


Figure 31: Example of TriggerCommunication settings for the CheckPresence recipe.